Instituto Superior
de Engenharia
Universidade Politécnica de Coimbra

Sockets TCP

Introdução às Redes de Comunicação

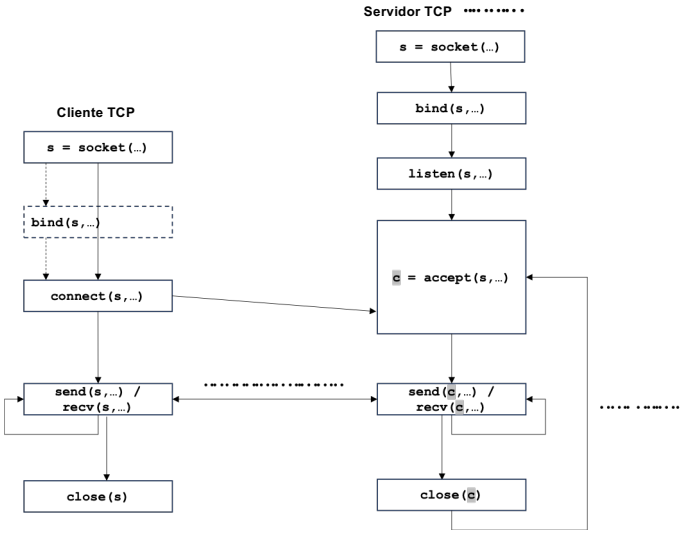
Licenciatura em Engenharia Informática

Sockets

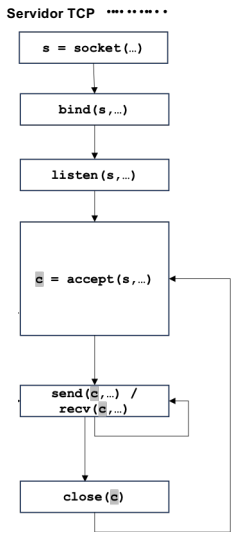
- Permite a troca de mensagens entre os processos de uma máquina/aplicação servidor e de uma máquina/aplicação cliente
- Tipos de Sockets:
  - **STREAM (TCP) fornecem garantia na entrega**
  - DATAGRAM (UDP) são mais simples e sem garantia
- Conexões definidas por porto (0 - 65535)
- Servidor pode suportar múltiplas conexões simultâneas (mais de 1 cliente na mesma porta)
  - Simples de implementar em DATAGRAM (UDP)
  - Necessário usar *threads* em STREAM (TCP)

Igual ao UDP

Fluxo de *sockets* TCP

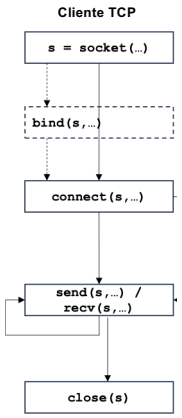


Implementação – servidor TCP



- **socket()**
  - Define o tipo de *socket*: stream (TCP)
- **bind()**
  - Especifica o endereço (IP e porto) por onde serão esperadas ligações
- **listen()**
  - Configura o *socket* em modo escuta
- **accept()**
  - Aguarda por ligação de cliente
  - A ligação é estabelecida através de um **novo descritor *socket***
  - Para múltiplas ligações → múltiplos descritores *socket*
- **send() / recv()**
  - Realiza a troca de mensagens através do novo descritor socket
- close() → em Winsock é a função **closesocket()**
  - Fecha o socket

Implementação – cliente TCP



- **socket()**
  - Define o tipo de *socket*: stream (TCP)
- **connect()**
  - Estabelece a ligação com o servidor
- **send() / recv()**
  - Realiza a troca de mensagens
- close() → em Winsock é a função **closesocket()**
  - Fecha o socket

Sockets em Windows (Winsock)

```
SOCKET socket(int af, int type, int protocol);
```

- **`af`** (Address Family)
  - Família de endereços (ou domínio) da comunicação. Define o formato dos endereços de rede que serão usados:
    - `AF_INET`: Endereçamento IPv4 (32 bits)
    - `AF_INET6`: Endereçamento IPv6 (128 bits)
    - `AF_BTH`: Bluetooth (suportado nativamente no Winsock)
- **`type`** (Socket Type)
  - Protocolo da transmissão dos dados:
    - **`SOCK_STREAM`**: Fornece fluxos de bytes bidirecionais, fiáveis e baseados em ligação (orientado à ligação). Garante entrega por ordem, sem duplicação e sem limites de mensagens. Usado com TCP
    - `SOCK_DGRAM`: Fornece datagramas sem ligação (*connectionless*), não fiáveis e de comprimento fixo máximo. Não garante ordem nem entrega. Usado com UDP
    - `SOCK_RAW`: Acesso direto ao nível do protocolo IP (permite manipular ou inspecionar cabeçalhos de rede; no Windows exige privilégios de administrador)
- **`protocol`** (Protocol)
  - Protocolo de transporte concreto. Na grande maioria dos casos passa-se 0, permitindo que o SO escolha o protocolo por omissão para o par (af, type). Pode tomar os valores:
    - `IPPROTO_TCP`: Para ligações TCP (af = AF_INET/AF_INET6, type = SOCK_STREAM)
    - `IPPROTO_UDP`: Para datagramas UDP (af = AF_INET/AF_INET6, type = SOCK_DGRAM)
    - `IPPROTO_RAW`: Para *sockets* em modo bruto

Igual ao UDP

Sockets em Windows (Winsock)

```
int bind(SOCKET s, const struct sockaddr *name, int namelen);
```

Igual ao UDP

- **s**
  - Descritor do *socket* retornado por `socket()`
- **name**
  - Apontador genérico para a estrutura com os detalhes de rede
    - Contém a informação do **IP** e do **porto**
  - Na prática, instancia-se uma estrutura específica para a família pretendida:
    `struct sockaddr_in` para IPv4
    `struct sockaddr_in6` para IPv6
- **namelen**
  - Dimensão em *bytes* da estrutura de endereço passada
    - ex: `sizeof(struct sockaddr_in)`

Sockets em Windows (Winsock)

```
struct sockaddr_in {
    short          sin_family;      // Familia de endereços (AF_INET)
    u_short        sin_port;        // Número do porto (em Network Byte Order)
    struct in_addr sin_addr;        // Endereço IP de 32 bits (em Network Byte Order)
    char           sin_zero[8]; };  // Bytes de enchimento para alinhar a 16 bytes
```

Igual ao UDP

- **sin_family**
  - Especifica a família de endereços. Deve conter o valor `AF_INET`
- **sin_port**
  - O porto de comunicação TCP ou UDP (de 0 a 65535)
  - Tem de ser obrigatoriamente convertido *Network Byte Order* (*Big-Endian*) usando a macro `htons()` (*host to network short*)
- **sin_addr**
  - O endereço IPv4. Também tem de estar em *Network Byte Order*. Pode ser configurado através de:
    - `htonl(INADDR_ANY)`: No caso de um servidor que deva escutar em todas as interfaces locais (0.0.0.0)
    - `inet_pton(AF_INET, "192.168.1.10", &addr.sin_addr)`: Abordagem recomendada para converter uma cadeia de carateres com notação decimal pontuada para o formato binário
      - `addr.sin_addr.s_addr = inet_addr("192.168.1.10")`: A evitar, não verifica se o IP está correto!

Sockets em Windows (Winsock)

```
int listen(SOCKET s, int backlog);
```

- **s**
  - Descritor do *socket* retornado por `socket()`
- **backlog**
  - Tamanho máximo da fila para ligações em espera
    - Usar o valor **SOMAXCONN**, automaticamente gerido pelo Sistema Operativo

Sockets em Windows (Winsock)

```
SOCKET accept(SOCKET s, struct sockaddr *addr, int *addrlen);
```

- **s**
  - Descritor do *socket* retornado por `socket()`
- **addr**
  - Apontador genérico para a estrutura com os detalhes de rede
    - Contém a informação do **IP** e do **porto**
  - Na prática, instancia-se uma estrutura específica para a família pretendida:
    `struct sockaddr_in` para IPv4
    `struct sockaddr_in6` para IPv6
- **addrlen**
  - Apontador para a variável com a dimensão em bytes da estrutura de endereço passada em addr
- Retorna um novo descritor *socket* a usar para a troca de mensagens

Sockets em Windows (Winsock)

```
int connect(SOCKET s, struct sockaddr *name, int namelen);
```

- **s**
  - Descritor do *socket* retornado por `socket()`
- **name**
  - Apontador genérico para a estrutura com os detalhes de rede
    - Contém a informação do **IP** e do **porto**
  - Na prática, instancia-se uma estrutura específica para a família pretendida:
    `struct sockaddr_in` para IPv4
    `struct sockaddr_in6` para IPv6
- **namelen**
  - Dimensão em *bytes* da estrutura de endereço passada
    - ex: `sizeof(struct sockaddr_in)`

Sockets em Windows (Winsock)

```
int recv(SOCKET c, const char *buf, int len, int flags);

int send(SOCKET c, char *buf, int len, int flags);
```

- **c** (descritor *socket*)
  - Descritor do *socket* retornado por `accept()`
- **buf** (buffer de dados)
  - Apontador para a zona de memória contendo os *bytes* da mensagem
- **len** (comprimento dos dados)
  - Número de *bytes* de `buf`
- **flags** (modificadores de comportamento)
  - Ajusta a forma como o envio é processado pela pilha de rede. Na maioria dos casos passa-se 0

Nota: Em TCP não é necessária a informação do destinatário/remetente, dado que foi inicialmente estabelecida a ligação

Obtenção do IP a partir do nome

```
int getaddrinfo(char *node, char *service, const struct addrinfo *hint,
                struct addrinfo *result);
```

- **node**
  - *String* com o nome a converter em IP
- **service**
  - *String* com o nome do serviço ou porto
    - Para a obtenção do IP da partir do nome, passa-se `NULL`
- **hint**
  - Ponteiro para estrutura com informação sobre o tipo de socket suportado
    - Para a obtenção do IP da partir do nome, passa-se `NULL`
- **result**
  - Ponteiro para lista ligada de 1 ou mais `struct addrinfo` com informação do resultado

Nota: se for passado um IP em `node`, o resultado é o IP

Obtenção do IP a partir do nome

```
struct addrinfo {
  int              ai_flags;
  int              ai_family;          // Familia de endereços
  int              ai_socktype;
  int              ai_protocol;
  int              ai_addrlen;
  char             *ai_canonname;
  struct sockaddr  *ai_addr;           // Estrutura com a informação resultado
  struct addinfo   *ai_next;           // Próximo elemento da lista
};
```

Pode haver mais do que um resultado para o mesmo nome

- Percorrer a lista ligada e aproveitar apenas o da família correta (se IPv4 → AF_INET)

```
while(result != NULL) {
    if (result->ai_family == AF_INET)
        addr.sin_addr = ((struct sockaddr_in*)result->ai_addr)->sin_addr;
    result = result->ai_next;
}
```